

Day 5 — Statistics, Experimentation, and Causal Reasoning

Beginner-friendly summary

Statistical reasoning helps you answer three questions:

1. **What happened?** Use descriptive statistics and exploratory data analysis.
2. **Could the observed difference be random?** Use confidence intervals, hypothesis tests, and power analysis.
3. **Did the intervention cause the difference?** Use randomization, causal reasoning, and careful study design.

A senior applied scientist should never stop at “the p-value is below 0.05.” A strong conclusion includes:

- The estimated effect
- Its uncertainty
- Whether it is economically meaningful
- Whether the data and experiment are trustworthy
- Whether the result is causal
- What decision should be made under risk

All numbers in the practical exercise below are **synthetic and intended only for interview preparation**.

```
Business question
  |
  v
Define population, unit, metric, estimand
  |
  v
Validate data --> missingness, anomalies, leakage, reconciliation
  |
  v
Study design --> randomization, controls, sample size
  |
  v
EDA --> distributions, segments, temporal patterns
  |
  v
Estimate effect + confidence interval + hypothesis test
  |
  v
Robustness, causal limitations, business significance
  |
  v
Decision, risks, and recommended next action
```

1. Descriptive statistics and distributions

Descriptive statistics summarize the data you observed. They do not, by themselves, tell you whether the pattern generalizes or is causal.

Measures of location

Measure	Meaning	Best used when
Mean	Arithmetic average	Distribution is reasonably symmetric; totals matter
Median	Middle observation	Data is skewed or contains extreme values
Mode	Most frequent value	Categories or repeated discrete values
Percentile	Value below which a percentage falls	Tail-risk, SLA, latency, loss, and spend analysis

For monthly expenses:

- Mean expense may be ₹50,000.
- Median may be ₹30,000.
- The difference suggests a right-skewed distribution caused by a few large expenses.
- The 95th percentile may be ₹140,000, which is important for liquidity or exception-management decisions.

Measures of spread

Range

$$\text{Range} = \max(X) - \min(X)$$

It is easy to understand but highly sensitive to outliers.

Variance

Population variance:

$$\sigma^2 = E[(X - \mu)^2]$$

Sample variance:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

The $n - 1$ denominator corrects the tendency of a sample to underestimate population variance.

Standard deviation

$$s = \sqrt{s^2}$$

It is expressed in the original units and measures typical dispersion around the mean.

Interquartile range

$$IQR = Q_{75} - Q_{25}$$

A common outlier-screening rule is:

$$x < Q_1 - 1.5IQR$$

or

$$x > Q_3 + 1.5IQR$$

This is an investigation rule, not an automatic deletion rule.

Distribution shape

Symmetric distribution

Mean and median are usually close.

Right-skewed distribution

A small number of high observations pull the mean upward.

Common examples:

- Invoice values
- Insurance losses
- Transaction amounts
- Customer lifetime value
- Request latency

Possible analytical responses include:

- Report median and percentiles
- Apply a log transformation
- Use robust or non-parametric methods
- Model the distribution explicitly
- Winsorize only under a justified, pre-specified rule

Never remove large financial values simply because they are inconvenient. They may represent fraud, duplicate payments, real tail risk, or a data defect.

Covariance

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- Positive covariance: the variables tend to increase together.
- Negative covariance: one tends to decrease as the other increases.
- Near-zero covariance: little linear co-movement.

Its magnitude depends on the variables' units, making it difficult to compare across variable pairs.

Correlation

Pearson correlation standardizes covariance:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{s_X s_Y}$$

It lies between -1 and 1 .

Important limitations:

- It measures linear association.
- Outliers can strongly affect it.
- Zero correlation does not imply independence.
- High correlation does not establish causation.
- Time trends can create spurious correlation.

For non-linear or ordinal relationships, Spearman rank correlation may be more appropriate.

2. Conditional probability, Bayes, expected value, and uncertainty

Conditional probability

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

This asks: “What is the probability of A , given that B has occurred?”

For example:

- A : a transaction is fraudulent
- B : the fraud detector raises an alert

The useful probability is usually $P(\text{fraud} | \text{alert})$, not merely the detector’s sensitivity.

Bayes’ theorem

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Suppose:

- Fraud prevalence: 1%
- Detector sensitivity: 90%
- False-positive rate: 5%

Then:

$$P(\text{fraud} | \text{alert}) = \frac{0.90 \times 0.01}{0.90 \times 0.01 + 0.05 \times 0.99} \approx 15.4\%$$

Although the model detects 90% of fraudulent transactions, most alerts may still be false because fraud is rare.

This is the **base-rate effect**, a common applied-science interview topic.

Expected value

For possible outcomes x_i with probabilities p_i :

$$E[X] = \sum_i p_i x_i$$

Suppose a review action:

- Prevents a ₹100,000 loss with probability 10%
- Costs ₹2,000 to perform

Expected benefit:

$$0.10 \times 100,000 = ₹10,000$$

Expected net value:

$$₹10,000 - ₹2,000 = ₹8,000$$

A decision can have positive expected value even when success is not guaranteed.

Variance and decision risk

Expected value alone is insufficient.

Two investments may have the same expected return but very different variance and downside risk. Finance stakeholders may care about:

- Probability of loss
- Worst-case exposure
- Value at risk
- Cash-flow volatility
- Tail scenarios
- Confidence that a threshold will be exceeded

A senior answer should therefore distinguish:

- Expected outcome
- Outcome variability
- Downside risk
- Uncertainty in the estimate itself

3. Sampling and bias

A statistically sophisticated test cannot repair an unrepresentative dataset.

Selection bias

Selection bias occurs when inclusion in the dataset is related to the outcome.

Example:

You evaluate an expense-management feature only among departments that voluntarily enabled it. Departments with strong budget discipline may be more likely to enable the feature.

The feature group could appear better even if the feature had no effect.

Survivorship bias

You analyze only entities that remain active.

Examples:

- Evaluating investments using only surviving funds
- Studying successful vendors while excluding terminated vendors
- Evaluating loan performance using only accounts that remain open
- Measuring model quality only on requests that completed successfully

Failures and exits often contain the most important information.

Non-response bias

People who respond may differ systematically from those who do not.

For example, employees with strong opinions may be more likely to answer a finance-tool satisfaction survey.

A large sample does not eliminate this bias.

Representative datasets

A representative dataset should reflect the deployment population across relevant dimensions:

- Region
- Time period
- Department type
- Transaction size
- Customer segment
- Product type
- Seasonal periods
- High-risk and low-risk entities
- Successful and failed cases

For temporal ML systems, random row-level splitting may create an unrealistic dataset. A future-period holdout is often more representative of production.

4. Confidence intervals

A confidence interval provides a range of values compatible with the observed data and statistical procedure.

Suppose an estimated treatment effect is:

−3.25 percentage points

with a 95% confidence interval:

$[-4.61, -1.90]$

The operational interpretation is:

| The data supports a reduction somewhere around 1.90 to 4.61 percentage points, assuming the sampling, independence, and model assumptions are valid.

What a frequentist 95% confidence interval means

Across many hypothetical repetitions of the same procedure, approximately 95% of the resulting intervals would contain the true population parameter.

It does **not technically mean**:

| There is a 95% probability that this particular fixed interval contains the true parameter.

That probability statement belongs more naturally to a Bayesian credible interval, conditional on its model and prior.

What a confidence interval does not protect against

A narrow interval can still be misleading when there is:

- Selection bias
- Leakage
- Confounding
- Incorrect randomization
- Measurement error
- Sample-ratio mismatch
- Unmodeled clustering
- Multiple testing
- Incorrect data reconciliation

Confidence intervals quantify sampling uncertainty under assumptions. They do not automatically quantify all sources of business uncertainty.

5. Hypothesis testing

Null and alternative hypotheses

Suppose a budget-alert system is intended to reduce department overspending.

The null hypothesis might be:

$$H_0 : \mu_{\text{alert}} - \mu_{\text{control}} = 0$$

The alternative:

$$H_1 : \mu_{\text{alert}} - \mu_{\text{control}} \neq 0$$

A two-sided alternative is generally safer unless directionality was justified and pre-specified before seeing the data.

P-value

A p-value is:

The probability, assuming the null hypothesis and test assumptions are true, of observing a result at least as extreme as the result obtained.

It is not:

- The probability that the null hypothesis is true
- The probability the result happened “by chance”
- The probability the experiment will reproduce
- The size of the business impact

Type I error

A Type I error is a false positive:

- You conclude the intervention has an effect.
- In reality, it does not.

Its probability is controlled by α , commonly 0.05.

Type II error

A Type II error is a false negative:

- You fail to detect an effect.
- A meaningful effect really exists.

Its probability is β .

Power is:

$$1 - \beta$$

A common target is 80% or 90% power.

Failing to reject is not proving no effect

A non-significant result could mean:

- There is no effect.
- The effect is too small to detect.
- The sample is too small.
- The variance is too high.
- The metric is noisy.
- The experiment is poorly implemented.

For an equivalence or non-inferiority claim, use a test designed for that purpose rather than interpreting $p > 0.05$ as equality.

6. Selecting the statistical test

Situation	Common method	Key considerations
Two independent continuous groups	Welch two-sample t-test	Prefer over pooled t-test when variances may differ
Same units before and after	Paired t-test	Analyze within-unit differences
Binary outcome in two large groups	Two-proportion z-test	Expected counts must be sufficiently large
Binary outcome with small counts	Fisher's exact test	Exact but may be conservative
Association between categories	Chi-square test	Requires adequate expected cell counts
Distributional assumptions uncertain	Permutation test	Requires exchangeability under the null
Interval for a complex statistic	Bootstrap	Resampling units must match the sampling design
Highly skewed or ordinal outcome	Mann–Whitney U	Tests rank/distribution differences, not automatically medians
Multiple related observations per unit	Mixed model, GEE, cluster analysis	Ordinary tests underestimate uncertainty
Time series or interrupted rollout	Time-series or causal time methods	Observations are autocorrelated

Welch t-test

Welch's test compares group means without assuming equal variances.

It is usually a better default than the classical pooled-variance t-test.

Conditions:

- Independent experimental units

- Continuous outcome
- Means are scientifically meaningful
- No extreme unresolved data errors
- Sampling distribution of the mean is reasonably stable

The raw observations do not have to be perfectly normally distributed, particularly with moderate samples, but severe skew and influential observations require sensitivity checks.

Proportion test

Use when the metric is binary, such as:

- Approval versus rejection
- Fraud versus non-fraud
- Late versus on-time
- Conversion versus no conversion

Possible effect measures:

- Absolute risk difference
- Relative risk
- Odds ratio

For stakeholders, absolute risk difference is often easiest to interpret.

Chi-square test

Used for association between categorical variables.

Example:

Group	Late	On time
Control	80	920
Treatment	55	945

Chi-square evaluates whether group and outcome are independent.

Permutation test

The procedure:

1. Calculate the observed treatment-control difference.
2. Shuffle group labels.
3. Recalculate the difference.
4. Repeat many times.
5. Compare the observed difference with the shuffled null distribution.

It is particularly useful when:

- The randomization design supports label exchangeability.

- You want fewer parametric assumptions.
- The statistic is unusual.

The permutation must respect the original design. For cluster randomization, shuffle clusters—not individual rows.

Bootstrap

The bootstrap estimates uncertainty by repeatedly resampling observed units with replacement.

It is useful for:

- Confidence intervals
- Medians
- Quantiles
- Ratios
- Complex metrics
- Model-performance differences

For repeated transactions within customers, resampling transaction rows independently would be wrong. Resample customers or clusters.

7. Statistical significance versus business significance

A sufficiently large sample can make a tiny, unimportant effect statistically significant.

Conversely, a meaningful effect may fail to reach statistical significance in a small study.

Always report:

1. Point estimate
2. Confidence interval
3. Effect size
4. Business threshold
5. Operational cost and risk

Common effect sizes

Mean difference

$$\Delta = \bar{x}_T - \bar{x}_C$$

Most interpretable when units have direct business meaning.

Standardized mean difference

Cohen's d :

$$d = \frac{\bar{x}_T - \bar{x}_C}{s_{\text{pooled}}}$$

Hedges' g applies a small-sample correction.

Approximate historical conventions:

- 0.2: small
- 0.5: medium
- 0.8: large

These are not universal business thresholds. Domain economics should determine meaningfulness.

Absolute risk difference

$$P(Y = 1 | T) - P(Y = 1 | C)$$

Relative risk

$$\frac{P(Y = 1 | T)}{P(Y = 1 | C)}$$

A relative reduction can sound impressive while representing a tiny absolute change, so both should be reported.

8. Power, minimum detectable effect, and sample size

Power is the probability of detecting a specified effect when it truly exists.

Power increases with:

- Larger sample size
- Larger true effect
- Lower variance
- Higher significance threshold
- Better experimental design
- Better variance-reduction methods

For two equal-sized independent groups, a rough sample-size approximation is:

$$n_{\text{per group}} \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\delta^2}$$

Where:

- δ : minimum detectable difference
- σ : outcome standard deviation
- α : Type I error rate
- $1 - \beta$: desired power

Using standardized effect $d = \delta/\sigma$, 5% two-sided significance, and 80% power:

$$n_{\text{per group}} \approx \frac{15.7}{d^2}$$

Approximate examples:

Standardized effect	Approximate sample per group
0.3	175
0.5	63
0.8	25

These are planning approximations, not substitutes for a design-specific power calculation.

Minimum detectable effect

The MDE is the smallest effect the planned experiment can reliably detect.

A useful stakeholder conversation is:

“With the available sample, we can reliably detect a reduction of at least 2 percentage points. We cannot distinguish smaller improvements from noise.”

This is better than discovering after the experiment that it was incapable of answering the business question.

Clustering

When departments are randomized but invoices are analyzed, invoice observations are not independent.

A rough design-effect correction is:

$$DE = 1 + (m - 1)\rho$$

Where:

- m : average observations per cluster
- ρ : intra-cluster correlation

Effective sample size is reduced approximately by the design effect.

9. A/B test design

Define the experiment precisely

Before launch, specify:

- Population
- Eligibility
- Randomization unit
- Treatment
- Control
- Primary metric

- Guardrail metrics
- Exposure logic
- Analysis period
- Minimum runtime
- Stopping rule
- Exclusion rules
- Statistical method
- Practical significance threshold

Randomization unit

Choose the unit at which treatment can be assigned without contamination.

Examples:

Intervention	Possible randomization unit
User interface banner	User
Department budget alert	Department
Vendor payment policy	Vendor or organization
Regional pricing rule	Region
Fraud-rule deployment	Account, merchant, or time block

If a department-wide alert affects everyone, randomizing individual expense claims would create contamination and invalid independence assumptions.

Primary metric

Choose one primary success metric.

For a budget-alert experiment:

$$\text{Overspend rate} = \frac{\text{Actual spend} - \text{Budget}}{\text{Budget}}$$

A normalized rate may be preferable to raw overspend when department budgets vary substantially.

Guardrail metrics

A treatment may improve the primary metric while harming the organization.

Potential guardrails:

- Payment delays
- Rejected valid expenses
- Employee support tickets
- Approval time
- Vendor complaints

- Month-end reconciliation errors
- Missing documentation
- System latency or failures

Novelty effects

Users may initially respond strongly to a new alert, but the response may fade.

Use:

- Adequate duration
- Week-by-week effect plots
- Post-novelty analysis
- Long-term holdouts where appropriate

Sample-ratio mismatch

If a 50/50 experiment produces 60/40 assignment or exposure, investigate before interpreting outcomes.

Possible causes:

- Broken randomization
- Eligibility differences
- Logging loss
- Treatment delivery failure
- Bot filtering
- Post-assignment exclusions

An SRM test commonly compares observed assignment counts with expected counts using chi-square.

A strong outcome p-value does not compensate for unresolved SRM.

Peeking

Repeatedly testing every day and stopping when $p < 0.05$ inflates the false-positive rate.

Safer approaches:

- Pre-specify sample size and duration
- Use group-sequential boundaries
- Use alpha-spending methods
- Use always-valid sequential methods
- Use Bayesian decision rules designed in advance

10. Multiple comparisons and sequential awareness

If you test 10 independent metrics at $\alpha = 0.05$, the probability of at least one false positive under all nulls is approximately:

$$1 - (1 - 0.05)^{10} \approx 40\%$$

Bonferroni correction

Use:

$$\alpha^* = \frac{\alpha}{m}$$

It controls family-wise error but can be conservative.

False discovery rate

Methods such as Benjamini–Hochberg control the expected proportion of false discoveries among rejected hypotheses.

FDR is useful when exploring many:

- Segments
- Features
- Model candidates
- Financial anomalies
- Biomarkers or scientific hypotheses

Good experimentation hierarchy

- One pre-specified primary metric
- A small set of guardrails
- Secondary metrics labeled supportive
- Exploratory segment results clearly labeled exploratory
- Confirmatory follow-up for unexpected findings

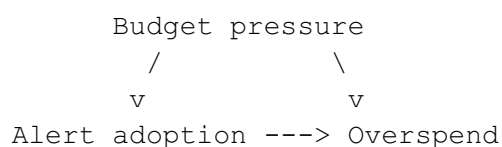
11. Causal reasoning basics

Correlation versus causation

Correlation means variables move together.

Causation means changing one variable would change the outcome under a well-defined intervention.

Suppose departments voluntarily adopt budget alerts:



Budget pressure affects both:

- Whether a department adopts alerts

- How much it overspends

It is a confounder.

Comparing voluntary adopters with non-adopters may therefore be biased.

Randomization

Random assignment makes treatment independent of measured and unmeasured pre-treatment characteristics in expectation.

It does not guarantee that every observed sample is perfectly balanced, but it makes remaining imbalance attributable to random variation rather than systematic selection.

Treatment effect

Potential-outcomes notation:

- $Y_i(1)$: unit i 's outcome under treatment
- $Y_i(0)$: unit i 's outcome under control

Individual treatment effect:

$$Y_i(1) - Y_i(0)$$

But only one potential outcome is observed for each unit. This is the fundamental causal-inference problem.

Average treatment effect:

$$ATE = E[Y(1) - Y(0)]$$

Randomized experiments estimate this by comparing treatment and control averages under appropriate conditions.

Important causal assumptions

Consistency

The treatment must be well-defined. "Received an alert" should not describe radically different interventions across units.

No interference

One department's treatment should not affect another department's outcome.

This may fail if departments share budgets, managers, or policies.

Correct assignment and exposure

Analyze assignment consistently, usually with intention-to-treat as the primary analysis.

No post-treatment conditioning

Do not control for variables caused by treatment unless the causal estimand specifically requires it.

For example, excluding departments that “did not engage with the alert” may introduce selection bias because engagement occurs after assignment.

12. Finance-oriented exploratory data analysis

Finance EDA must establish both statistical usability and accounting correctness.

Data structure and grain

First establish:

- What does one row represent?
- Is the row an invoice, department-month, vendor-month, or customer?
- What is the primary key?
- Are rows independent?
- Could the same transaction appear more than once?

A test at the wrong grain is often more dangerous than choosing the wrong test.

Missingness

Classify missing values:

- **MCAR:** missing unrelated to observed or unobserved values
- **MAR:** missing related to observed variables
- **MNAR:** missing related to the missing value itself

Examples:

- Vendor category missing because an older system did not capture it: potentially MAR.
- High-risk expenses deliberately omit documentation: potentially MNAR.

Investigate missingness by:

- Group
- Period
- Region
- Source system
- Amount
- Treatment status
- Outcome

Do not automatically mean-impute finance outcomes. Mean imputation distorts variance and relationships.

Anomalies

Check for:

- Duplicate invoices

- Reversed transactions
- Negative amounts
- Impossible dates
- Currency mismatches
- Extreme unit prices
- Budget equal to zero
- Actual spend many times larger than budget
- Unusual month-end postings
- Vendor-account combinations not previously seen

Anomalies should be categorized as:

1. Genuine high-value events
2. Fraud or abuse candidates
3. Data-entry errors
4. Integration or extraction defects
5. Accounting adjustments

Leakage

Leakage occurs when a feature contains information unavailable at prediction time.

Examples:

- Using final paid amount to predict whether an invoice will be approved
- Using a collections status updated after default
- Using month-end reconciled totals to make an intra-month forecast
- Using manually corrected fraud labels created after investigation

Always ask:

| “At what exact timestamp would this variable have been known?”

Temporal effects

Look for:

- Month-end and quarter-end spikes
- Fiscal-year boundaries
- Holidays
- Policy changes
- Inflation
- Vendor contract renewals
- Delayed posting
- Seasonality
- Data-pipeline changes

A treatment-control comparison can be biased if groups are observed over different periods.

Reconciliation checks

Useful invariants include:

$$\text{Opening balance} + \text{Credits} - \text{Debits} = \text{Closing balance}$$

$$\text{Total header amount} = \sum \text{line-item amounts}$$

$$\text{General ledger total} = \text{subledger total} + \text{approved adjustments}$$

Statistical analysis should not proceed while material reconciliation gaps remain unexplained.

13. Communicating uncertainty to finance stakeholders

Avoid saying:

| “The feature works because the p-value is significant.”

Prefer:

| “The pilot estimates a 3.25-percentage-point reduction in overspending. The plausible range under the study assumptions is approximately 1.90 to 4.61 percentage points. The estimated effect exceeds the pre-agreed business threshold, but the pilot covered only 60 departments and one period, so we recommend a monitored expansion rather than an immediate organization-wide assumption of the same impact.”

A strong executive explanation contains:

- What changed
 - How large the change was
 - The plausible range
 - Whether it clears the business threshold
 - The major risk to interpretation
 - What decision is recommended
 - What would change that recommendation
-

Practical task: synthetic budget-alert experiment

Business problem

A finance team introduces automated budget alerts for departments approaching their monthly budget.

The business question is:

| Do automated budget alerts reduce department-level overspending?

The experiment contains 60 departments:

- 30 assigned to control
- 30 assigned to budget alerts

The department is the randomization and analysis unit.

Analyzing individual invoices would be incorrect because invoices within the same department share managers, approval behavior, and budget pressure.

Dataset fields

Field	Description
department_id	Unique department
group	control or alert
budget	Monthly department budget
reported_spend	Initially reported spend
prior_period_spend	Spend before the pilot
invoice_count	Number of expense invoices
missing_vendor_pct	Percentage with missing vendor metadata
late_posting_pct	Percentage posted late
ledger_total	Independently extracted ledger total
duplicate_invoice_amount	Verified duplicate amount
reconciled_spend	Spend after approved corrections
overspend_rate	Primary outcome

Hypotheses

Primary null hypothesis:

$$H_0 : \mu_{\text{alert}} - \mu_{\text{control}} = 0$$

Alternative:

$$H_1 : \mu_{\text{alert}} - \mu_{\text{control}} / 0 =$$

A one-sided alternative could be used only if it were pre-specified and an increase in overspending would not require a different decision framework. A two-sided test is used here.

Primary metric

$$\text{Overspend rate} = \frac{\text{Reconciled spend} - \text{Budget}}{\text{Budget}}$$

Reasons for using a rate:

- Department budgets have different sizes.
- Raw overspend amounts would partly reflect budget scale.
- The rate directly measures deviation from the approved budget.

Secondary and guardrail metrics

Potential secondary metrics:

- Overspend amount
- Percentage of departments exceeding budget
- 90th percentile overspend
- Variance in overspending

Potential guardrails:

- Late-posting percentage
- Missing vendor metadata
- Invoice rejection rate
- Approval delay
- Support tickets

Only the overspend rate is treated as the primary confirmatory metric.

Analysis reasoning

The analysis should follow these decisions:

1. Verify that the department is the experimental unit.
 2. Confirm a 30/30 assignment split.
 3. Check unique department IDs.
 4. Review missing values.
 5. Reconcile reported spend against financial records.
 6. Investigate extreme overspend values rather than automatically deleting them.
 7. Calculate department-level overspend rates.
 8. Compare treatment and control distributions.
 9. Use Welch's t-test for the primary mean comparison.
 10. Report the mean difference and its 95% confidence interval.
 11. Report Hedges' g as a standardized effect size.
 12. Use bootstrap and permutation analyses as sensitivity checks.
 13. Separate statistical evidence from causal and operational limitations.
-

Pseudocode

```
SET deterministic random seed

GENERATE 60 synthetic department records
ASSIGN 30 departments to control
ASSIGN 30 departments to alerts

GENERATE:
    department budgets
    baseline spending
    invoice counts
    data-quality measures
    current overspend rates

INJECT known practice issues:
    one duplicated invoice
    one ledger reconciliation mismatch
    one missing guardrail value

CHECK:
    unique IDs
    assignment counts
    missing values
    invalid budgets
    ledger differences
    outliers

INVESTIGATE and resolve documented accounting issues
CALCULATE reconciled overspend rate

SPLIT department-level outcomes by experimental group

CALCULATE:
    group means and standard deviations
    treatment minus control mean difference
    Welch t-test
    Welch confidence interval
    Hedges g
    stratified bootstrap interval
    department-level permutation p-value

COMPARE parametric and resampling results

REPORT:
    point estimate
    interval
    statistical evidence
    business magnitude
    causal assumptions
    false-positive and scaling limitations
```

Python implementation

```
from __future__ import annotations

import numpy as np
import pandas as pd
from scipy import stats

RANDOM_SEED = 42
BOOTSTRAP_REPETITIONS = 20_000
PERMUTATION_REPETITIONS = 50_000

def create_synthetic_budget_data(seed: int = RANDOM_SEED) -> pd.DataFrame:
    """
    Create a deterministic synthetic department-level budget experiment.

    All values are artificial and intended for statistical practice.
    """
    rng = np.random.default_rng(seed)
    departments_per_group = 30
    total_departments = 2 * departments_per_group

    group = np.array(
        ["control"] * departments_per_group
        + ["alert"] * departments_per_group
    )

    budget = rng.integers(
        low=800_000,
        high=3_000_000,
        size=total_departments,
    )

    control_rate = rng.normal(
        loc=0.055,
        scale=0.035,
        size=departments_per_group,
    )
    alert_rate = rng.normal(
        loc=0.030,
        scale=0.035,
        size=departments_per_group,
    )

    generated_rate = np.concatenate([control_rate, alert_rate])
    generated_rate = np.clip(generated_rate, -0.04, 0.16)

    reported_spend = budget * (1 + generated_rate)

    data = pd.DataFrame(
        {
```



```

        "department_id": [
            f"D{i:02d}" for i in range(1, total_departments + 1)
        ],
        "group": group,
        "budget": budget,
        "reported_spend": reported_spend,
        "prior_period_spend": budget
        * (1 + rng.normal(0.05, 0.04, total_departments)),
        "invoice_count": rng.integers(
            80, 500, size=total_departments
        ),
        "missing_vendor_pct": np.round(
            rng.beta(1.2, 20, total_departments) * 100,
            2,
        ),
        "late_posting_pct": np.round(
            rng.beta(1.4, 12, total_departments) * 100,
            2,
        ),
        "region": rng.choice(
            ["North", "South", "East", "West"],
            size=total_departments,
        ),
    }
)

# Synthetic data-quality issue 1:
# A verified duplicate invoice inflated D08's reported spend.
data["duplicate_invoice_amount"] = 0.0
data.loc[7, "reported_spend"] += 420_000
data.loc[7, "duplicate_invoice_amount"] = 420_000

# Synthetic data-quality issue 2:
# A stale ledger extraction creates a temporary mismatch for D13.
data["ledger_total"] = data["reported_spend"]
data.loc[12, "ledger_total"] -= 55_000

# Synthetic missing guardrail value.
data.loc[43, "late_posting_pct"] = np.nan

return data

def perform_data_quality_checks(data: pd.DataFrame) -> dict[str, object]:
    if data["department_id"].duplicated().any():
        raise ValueError("Department IDs must be unique.")

    if (data["budget"] <= 0).any():
        raise ValueError("Budgets must be positive.")

    expected_groups = {"control", "alert"}
    observed_groups = set(data["group"].unique())

    if observed_groups != expected_groups:

```

```

        raise ValueError(
            f"Unexpected experiment groups: {observed_groups}"
        )

assignment_counts = data["group"].value_counts().sort_index()
missing_counts = data.isna().sum()

data = data.copy()
data["initial_ledger_gap"] = (
    data["reported_spend"] - data["ledger_total"]
)

reconciliation_issues = data.loc[
    ~np.isclose(data["initial_ledger_gap"], 0),
    [
        "department_id",
        "reported_spend",
        "ledger_total",
        "initial_ledger_gap",
    ],
]

return {
    "assignment_counts": assignment_counts,
    "missing_counts": missing_counts,
    "reconciliation_issues": reconciliation_issues,
}

def reconcile_spend(data: pd.DataFrame) -> pd.DataFrame:
    """
    Apply only documented accounting corrections.

    In production, corrections should come from an approved reconciliation
    process rather than from an analyst's arbitrary outlier rule.
    """
    reconciled = data.copy()

    reconciled["reconciled_spend"] = (
        reconciled["reported_spend"]
        - reconciled["duplicate_invoice_amount"]
    )

    # For this synthetic exercise, investigation confirms that:
    # - D08 contained a duplicate invoice.
    # - D13's ledger extract was stale, while the source transaction
    #   documents supported the reported amount.
    #
    # We model the resolved ledger after that investigation.
    reconciled["resolved_ledger_total"] = reconciled["ledger_total"]

    duplicate_mask = reconciled["duplicate_invoice_amount"] > 0
    reconciled.loc[
        duplicate_mask, "resolved_ledger_total"
    ]

```

```

] = reconciled.loc[duplicate_mask, "reconciled_spend"]

stale_extract_mask = reconciled["department_id"] == "D13"
reconciled.loc[
    stale_extract_mask, "resolved_ledger_total"
] = reconciled.loc[stale_extract_mask, "reconciled_spend"]

if not np.allclose(
    reconciled["reconciled_spend"],
    reconciled["resolved_ledger_total"],
):
    raise ValueError(
        "Material reconciliation differences remain unresolved."
    )

reconciled["overspend_rate"] = (
    reconciled["reconciled_spend"] - reconciled["budget"]
) / reconciled["budget"]

reconciled["overspend_amount"] = (
    reconciled["reconciled_spend"] - reconciled["budget"]
)

reconciled["prior_overspend_rate"] = (
    reconciled["prior_period_spend"] - reconciled["budget"]
) / reconciled["budget"]

return reconciled

def identify_iqr_outliers(
    values: pd.Series,
) -> tuple[pd.Series, float, float]:
    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    mask = (values < lower_bound) | (values > upper_bound)
    return mask, lower_bound, upper_bound

def welch_interval(
    treatment: np.ndarray,
    control: np.ndarray,
    confidence: float = 0.95,
) -> tuple[float, float, float, float]:
    treatment_mean = treatment.mean()
    control_mean = control.mean()
    difference = treatment_mean - control_mean

    treatment_variance = treatment.var(ddof=1)

```

```

control_variance = control.var(ddof=1)

treatment_n = len(treatment)
control_n = len(control)

standard_error = np.sqrt(
    treatment_variance / treatment_n
    + control_variance / control_n
)

numerator = (
    treatment_variance / treatment_n
    + control_variance / control_n
) ** 2

denominator = (
    (treatment_variance / treatment_n) ** 2
    / (treatment_n - 1)
    + (control_variance / control_n) ** 2
    / (control_n - 1)
)

degrees_of_freedom = numerator / denominator

alpha = 1 - confidence
critical_value = stats.t.ppf(
    1 - alpha / 2,
    df=degrees_of_freedom,
)

lower = difference - critical_value * standard_error
upper = difference + critical_value * standard_error

return difference, lower, upper, degrees_of_freedom

def hedges_g(
    treatment: np.ndarray,
    control: np.ndarray,
) -> float:
    treatment_n = len(treatment)
    control_n = len(control)

    treatment_variance = treatment.var(ddof=1)
    control_variance = control.var(ddof=1)

    pooled_variance = (
        (treatment_n - 1) * treatment_variance
        + (control_n - 1) * control_variance
    ) / (treatment_n + control_n - 2)

    pooled_standard_deviation = np.sqrt(pooled_variance)

    cohens_d = (

```

```

        treatment.mean() - control.mean()
    ) / pooled_standard_deviation

    correction = 1 - (
        3 / (4 * (treatment_n + control_n) - 9)
    )

    return correction * cohens_d

def bootstrap_mean_difference(
    treatment: np.ndarray,
    control: np.ndarray,
    repetitions: int = BOOTSTRAP_REPETITIONS,
    seed: int = 100,
) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    differences = np.empty(repetitions)

    for i in range(repetitions):
        treatment_sample = rng.choice(
            treatment,
            size=len(treatment),
            replace=True,
        )
        control_sample = rng.choice(
            control,
            size=len(control),
            replace=True,
        )

        differences[i] = (
            treatment_sample.mean()
            - control_sample.mean()
        )

    lower, upper = np.percentile(
        differences,
        [2.5, 97.5],
    )
    return float(lower), float(upper)

def permutation_test(
    treatment: np.ndarray,
    control: np.ndarray,
    repetitions: int = PERMUTATION_REPETITIONS,
    seed: int = 200,
) -> float:
    rng = np.random.default_rng(seed)

    observed_difference = abs(
        treatment.mean() - control.mean()
    )

```

```

combined = np.concatenate([treatment, control])
treatment_n = len(treatment)

extreme_count = 0

for _ in range(repetitions):
    shuffled = rng.permutation(combined)

    shuffled_treatment = shuffled[:treatment_n]
    shuffled_control = shuffled[treatment_n:]

    shuffled_difference = abs(
        shuffled_treatment.mean()
        - shuffled_control.mean()
    )

    if shuffled_difference >= observed_difference:
        extreme_count += 1

# The +1 correction avoids returning a p-value of exactly zero.
return (extreme_count + 1) / (repetitions + 1)

def main() -> None:
    raw_data = create_synthetic_budget_data()
    checks = perform_data_quality_checks(raw_data)

    print("Assignment counts:")
    print(checks["assignment_counts"])
    print()

    print("Non-zero missing-value counts:")
    missing = checks["missing_counts"]
    print(missing[missing > 0])
    print()

    print("Initial reconciliation issues:")
    print(checks["reconciliation_issues"])
    print()

    data = reconcile_spend(raw_data)

    outlier_mask, lower_bound, upper_bound = identify_iqr_outliers(
        data["overspend_rate"]
    )

    print(
        f"IQR bounds: {lower_bound:.4f} to {upper_bound:.4f}"
    )

    print("Potential outliers after reconciliation:")
    print(
        data.loc[
            outlier_mask,
            ["department_id", "group", "overspend_rate"],

```

```

    ]
)
print()

group_summary = data.groupby("group")[
    "overspend_rate"
].agg(
    ["count", "mean", "std", "median", "min", "max"]
)

print("Group summary:")
print(group_summary)
print()

treatment = data.loc[
    data["group"] == "alert",
    "overspend_rate",
].to_numpy()

control = data.loc[
    data["group"] == "control",
    "overspend_rate",
].to_numpy()

t_result = stats.ttest_ind(
    treatment,
    control,
    equal_var=False,
)

difference, ci_lower, ci_upper, welch_df = welch_interval(
    treatment,
    control,
)

standardized_effect = hedges_g(treatment, control)

bootstrap_lower, bootstrap_upper = (
    bootstrap_mean_difference(treatment, control)
)

permutation_p = permutation_test(treatment, control)

print(f"Alert mean: {treatment.mean():.4%}")
print(f"Control mean: {control.mean():.4%}")
print(f"Difference: {difference:.4%}")
print(
    "95% Welch CI: "
    f"[{ci_lower:.4%}, {ci_upper:.4%}]"
)

print(f"Welch degrees of freedom: {welch_df:.2f}")
print(f"Welch p-value: {t_result.pvalue:.8f}")
print(f"Hedges' g: {standardized_effect:.3f}")
print(

```

```

    "95% bootstrap interval: "
    f"[{bootstrap_lower:.4%}, {bootstrap_upper:.4%}]"
)
print(f"Permutation p-value: {permutation_p:.8f}")

monetary_summary = data.groupby("group") [
    "overspend_amount"
].mean()

print()
print("Mean overspend amount by group:")
print(monetary_summary)

if __name__ == "__main__":
    main()

```

Practical EDA findings

Data-quality findings

The synthetic EDA finds:

- 30 control departments
- 30 alert departments
- No duplicated department IDs
- One missing `late_posting_pct` value
- One initial ₹55,000 ledger mismatch
- One verified ₹420,000 duplicate invoice
- One remaining high—but valid—overspend-rate observation after reconciliation

The important distinction is:

- The duplicate invoice is corrected because documentary evidence identifies it as a duplicate.
- The remaining high-value department is not removed merely because it is an outlier.

The missing late-posting value affects a guardrail metric, not the primary overspending outcome. It should still be investigated, especially if missingness differs by treatment group.

Group-level descriptive results

After documented reconciliation:

Metric	Alert	Control
Departments	30	30
Mean overspend rate	2.64%	5.90%
Median overspend rate	2.69%	6.09%

Standard deviation	2.40 pp	2.82 pp
Minimum	-2.89%	0.40%
Maximum	6.00%	13.00%

The alert group has a lower mean and median.

The similar mean and median within each group suggest that the result is not being driven entirely by one extreme observation.

Statistical results

The estimated treatment-control difference is:

$$2.64\% - 5.90\% = -3.25 \text{ percentage points}$$

Results:

Result	Value
Mean difference	-3.25 percentage points
95% Welch interval	-4.61 to -1.90 percentage points
Welch p-value	Approximately 0.000011
Hedges' g	-1.23
Bootstrap interval	Approximately -4.56 to -1.98 percentage points
Permutation p-value	Approximately 0.00006

The parametric, bootstrap, and permutation analyses point in the same direction.

Hedges' $g = -1.23$ is a large standardized difference in this synthetic dataset. The business interpretation should nevertheless use the original percentage-point scale rather than relying on a generic "large effect" label.

The average budget across the synthetic departments is approximately ₹19.56 lakh. Applying the estimated 3.25-percentage-point difference gives an indicative amount of roughly:

$$₹19.56 \text{ lakh} \times 3.25\% \approx ₹63,600$$

per department per period.

This conversion is approximate. A production financial estimate should account for:

- Budget mix
- Treatment heterogeneity
- Implementation cost
- Alert operating cost
- Whether reduced spend is genuinely saved or merely delayed

- Downstream business impact
-

Why Welch's t-test was selected

Welch's test is appropriate because:

- The analysis unit is the independently randomized department.
- The outcome is continuous.
- The scientific question concerns the mean overspend rate.
- Group variances need not be assumed equal.
- There are 30 units in each group.
- Bootstrap and permutation checks are available for sensitivity analysis.

A paired test would be inappropriate because treatment and control departments are different units.

An invoice-level test would be inappropriate because it would treat correlated invoices as independent observations.

A proportion test would be appropriate only if the primary outcome were binary, such as whether the department exceeded its budget.

Correctness conditions

The causal interpretation depends on the following conditions.

Randomization was implemented correctly

Assignment must not have been changed after observing department characteristics.

Treatment assignment is the analysis basis

The primary analysis should normally be intention-to-treat:

- Departments are analyzed according to assignment.
- Departments should not be removed because they ignored the alerts.

Department outcomes are independent across departments

This may fail if:

- Departments share the same approving manager.
- One department can transfer budget to another.
- Alerts cause organization-wide policy changes.
- Departments communicate and copy behavior.

Outcomes are measured consistently

Treatment should not change how spend is recorded rather than how much is spent.

For example, departments might delay invoices into the next period. Overspending would appear lower without a genuine reduction in economic cost.

That is why late-posting and future-period spending are important guardrails.

Reconciliation rules are treatment-blind

Accounting corrections should not be more aggressive in one group than another.

Ideally, the reconciliation team should not know treatment assignment.

The primary metric was pre-specified

Selecting overspend rate only after observing that it produced the strongest result would increase false-positive risk.

False-positive and multiple-testing limitations

The primary p-value is small, but it should not be interpreted in isolation.

False-positive risk would be higher if analysts:

- Tested many expense metrics
- Examined many subgroup combinations
- Tried several outlier rules
- Changed the experiment window
- Repeatedly checked results
- Selected the most favorable result

For example, testing:

- Mean overspend
- Median overspend
- Overspend amount
- Budget-exceedance rate
- Invoice count
- Ten regional segments
- Multiple department-size segments

and reporting only the smallest p-value would invalidate the nominal 5% error rate.

The analysis should identify:

- Which result was confirmatory
 - Which results were sensitivity checks
 - Which findings were exploratory
-

Causal limitations

The randomized design supports a causal interpretation for the departments and period studied, assuming correct execution.

It does not automatically establish:

- Long-term persistence
- Effectiveness in other regions
- Effectiveness during year-end close
- Effectiveness for much larger departments
- The mechanism causing the change
- Organization-wide savings at the same rate

Possible alternative mechanisms include:

- Departments genuinely reducing unnecessary spend
- Departments postponing invoices
- Departments moving expenses to different cost centers
- Managers changing accounting classifications
- A temporary novelty response
- Differential missing-data behavior

A stronger follow-up would examine:

- Subsequent-period spending
- Late postings
- Cost-center transfers
- Vendor obligations
- Approval delays
- Treatment effect by department size
- Effect decay over time

Production trade-offs and failure modes

Mean versus median

The mean aligns with total financial impact but is sensitive to high values.

The median is robust but may hide costly tail events.

A finance analysis should usually report both and include tail percentiles.

Rate versus amount

Overspend rate normalizes departments of different sizes.

Overspend amount aligns more directly with currency impact.

A good analysis uses one as the primary metric and the other as supportive context rather than switching opportunistically.

Outlier removal

Removing extreme departments can reduce variance but may remove the exact risk finance cares about.

Use:

- Documented reconciliation
- Pre-specified data-quality rules
- Robust sensitivity analysis
- Analysis with and without influential but valid observations

Do not use:

- “Remove everything above three standard deviations” without investigation

Small number of clusters

Sixty departments can produce strong evidence for a large synthetic effect, but it may be insufficient for:

- Small effects
- Detailed subgroup analysis
- Estimating treatment heterogeneity
- Rare guardrail events

Temporal leakage

Do not use future reconciliation data when designing a real-time alert model unless that information would genuinely be available at alert time.

Delayed spending

A short experiment may confuse delayed spending with avoided spending.

A post-experiment observation window is necessary.

Treatment contamination

Managers controlling both treatment and control departments may apply alert-inspired practices to control departments, biasing the result toward zero.

Operational non-compliance

If many assigned departments never receive alerts because of system failures, report:

- Assignment effect: intention-to-treat
- Delivery rate
- Exposure diagnostics

A per-protocol result can be supportive but may be selection-biased.

Executive-friendly conclusion

In this synthetic pilot, departments assigned to automated budget alerts overspent by an average of 2.64%, compared with 5.90% for control departments. The estimated reduction was 3.25 percentage points, with a 95% confidence interval of approximately 1.90 to 4.61 percentage points. The result exceeds a plausible business threshold and was consistent across parametric, bootstrap, and permutation analyses.

The result supports a monitored expansion, but not an unconditional organization-wide savings assumption. Before full rollout, finance should confirm that the reduction represents avoided spending rather than delayed posting or cost-center shifting, monitor operational guardrails, and evaluate whether the effect persists over additional financial periods.

Recommended decision

Proceed with a controlled expansion when:

- The lower confidence bound exceeds the minimum economically useful reduction.
- Delayed postings and transfers remain within acceptable guardrails.
- Reconciliation issues are resolved consistently.
- Implementation costs are lower than conservative expected savings.
- The effect remains stable beyond the novelty period.

This is the central senior-level principle:

Make the decision using the estimated effect, uncertainty, economics, causal credibility, and downside risk—not the p-value alone.

Day 5 DSA — Sliding Window

Beginner-friendly summary

A **sliding window** maintains a contiguous section of an array or string while moving through the input.

Instead of recalculating information for every possible subarray or substring, we:

1. Add the new right-side element.
2. Remove elements from the left when necessary.
3. Maintain only the information needed for the current window.
4. Update the answer.

This often reduces an $O(n^2)$ brute-force solution to $O(n)$.

1. Recognition signals

Consider sliding window when the problem asks about a:

- Contiguous subarray
- Contiguous substring
- Consecutive sequence
- Maximum or minimum window
- Window of exactly size k
- Longest substring satisfying a condition
- Shortest subarray satisfying a condition
- Number of distinct elements within a range
- Frequency of characters inside a substring

Typical phrases include:

- “Longest substring...”
- “Smallest subarray...”
- “Maximum sum of k consecutive elements...”
- “At most k distinct characters...”
- “Without repeating characters...”
- “Contains all required characters...”

Sliding window is usually applicable when removing elements from the left can restore validity after extending the right boundary.

2. Fixed versus variable windows

Window type	Window size	Common examples
Fixed window	Always k	Maximum sum of k elements
Variable window	Expands and shrinks	Longest substring without duplicates
Frequency-map window	Tracks counts inside window	Anagrams, distinct characters, minimum window
Monotonic window	Maintains ordered candidates	Sliding-window maximum

3. Fixed-size sliding window

Example problem

Find the maximum sum of any k consecutive elements.

```
Input:  [2, 1, 5, 1, 3, 2]
k = 3
```

```
Windows:
[2, 1, 5] -> 8
[1, 5, 1] -> 7
[5, 1, 3] -> 9
[1, 3, 2] -> 6
```

Answer: 9

Brute-force reasoning

Generate every window of size k and calculate its sum independently.

There are approximately $n - k + 1$ windows, and each sum requires k operations.

$$O((n - k + 1)k) \approx O(nk)$$

Sliding-window reasoning

Calculate the first window sum once.

When moving one position:

- Subtract the element leaving the window.
- Add the element entering the window.

```
new_sum = old_sum - outgoing_element + incoming_element
```

This makes the total complexity $O(n)$.

Fixed-window template

```
def fixed_window(values: list[int], k: int) -> int:
    if k <= 0 or k > len(values):
        raise ValueError("k must be between 1 and len(values)")

    window_sum = sum(values[:k])
    best = window_sum

    for right in range(k, len(values)):
        left = right - k

        window_sum += values[right]
        window_sum -= values[left]

        best = max(best, window_sum)

    return best
```


4. Variable-size sliding window

A variable window grows while valid or while searching for a condition, then shrinks when necessary.

The most important skill is defining the **window invariant**.

What is an invariant?

An invariant is a condition that must remain true whenever we use the current window to update the answer.

Examples:

- Window contains no repeated characters.
- Window contains at most k distinct values.
- Window sum is at least the target.
- Window contains all required characters.
- Number of zeroes in the window is at most k .

The algorithm normally follows this structure:

```
for each right boundary:
    add the right element to the window

    while the window is invalid:
        remove the left element
        move left forward

    update the answer
```

Medium problem: Longest Substring Without Repeating Characters

Problem statement

Given a string s , return the length of the longest substring that contains no repeating characters.

Example 1

```
Input:  "abcabcbb"
Output: 3

Explanation:
"abc" is the longest substring without repeated characters.
```

Example 2

```
Input:  "bbbbbb"
Output: 1
```

Example 3

Input: "pwwkew"

Output: 3

Explanation:

"wke" is valid.

"pwwkew" is not a substring because its characters are not contiguous.

5. Recognition signals

The important signals are:

- The question asks for a **substring**, so elements must be contiguous.
- It asks for the **longest** valid range.
- Validity depends on characters currently inside the range.
- When a duplicate appears, removing characters from the left can restore validity.
- Character counts can be maintained incrementally.

Therefore, use a variable sliding window with a frequency map.

6. Brute-force reasoning

Generate every possible substring.

For each starting index:

1. Start an empty set.
2. Extend the ending index.
3. Stop when a duplicate is encountered.
4. Record the longest valid length.

There can be $O(n^2)$ substrings.

Checking duplicates with a set while extending each start still results in:

$$O(n^2)$$

Brute-force pseudocode

```
best = 0

for start from 0 to n - 1:
    seen = empty set

    for end from start to n - 1:
        if s[end] is already in seen:
            break

    add s[end] to seen
```

```
        best = max(best, end - start + 1)

return best
```

Brute-force implementation

```
def longest_unique_substring_brute_force(s: str) -> int:
    best = 0

    for start in range(len(s)):
        seen: set[str] = set()

        for end in range(start, len(s)):
            character = s[end]

            if character in seen:
                break

            seen.add(character)
            best = max(best, end - start + 1)

    return best
```

Complexity

- Time: $O(n^2)$
- Space: $O(m)$

Here, m is the number of distinct characters stored in the set.

7. Optimized reasoning

Maintain a window:

```
s[left : right + 1]
```

A frequency map records how many times each character appears in the current window.

For every `right`:

1. Add `s[right]`.
2. If its count becomes greater than one, the window is invalid.
3. Move `left` forward, reducing counts, until the duplicate is removed.
4. The window is now valid.
5. Update the maximum length.

Window invariant

After the shrinking loop finishes:

Every character appears at most once in `s[left:right + 1]`.

Because the window satisfies this invariant, its length can safely be considered for the answer.

8. Example walkthrough

Input:

```
s = "pwwkew"
```

Right	Added	Window before repair	Action	Valid window	Best
0	p	p	None	p	1
1	w	pw	None	pw	2
2	w	pww	Remove p, then first w	w	2
3	k	wk	None	wk	2
4	e	wke	None	wke	3
5	w	wkew	Remove first w	kew	3

Answer:

```
3
```

9. Optimized pseudocode

```
left = 0
best = 0
frequency = empty map

for right from 0 to length(s) - 1:
    current = s[right]
    increment frequency[current]

    while frequency[current] > 1:
        outgoing = s[left]
        decrement frequency[outgoing]
        left = left + 1

    window_length = right - left + 1
    best = max(best, window_length)

return best
```

10. Python solution

```
from collections import defaultdict

def length_of_longest_substring(s: str) -> int:
    """
    Return the maximum length of a substring containing no repeated
    characters.

    Time complexity: O(n)
    Space complexity: O(m), where m is the number of distinct characters.
    """
    frequency: dict[str, int] = defaultdict(int)

    left = 0
    best_length = 0

    for right, character in enumerate(s):
        frequency[character] += 1

        while frequency[character] > 1:
            outgoing_character = s[left]
            frequency[outgoing_character] -= 1
            left += 1

        current_length = right - left + 1
        best_length = max(best_length, current_length)

    return best_length
```

Example usage

```
print(length_of_longest_substring("abcabcbb")) # 3
print(length_of_longest_substring("bbbbbb"))  # 1
print(length_of_longest_substring("pwwkew"))  # 3
print(length_of_longest_substring(""))         # 0
```

11. Why the algorithm is $O(n)$

The nested `while` loop can make the algorithm appear to be $O(n^2)$, but it is not.

Each character:

- Enters the window once through `right`.
- Leaves the window at most once through `left`.

Both pointers move only forward.

Therefore, the total number of pointer movements is at most approximately $2n$:

$$O(n)$$

Complexity

Measure	Complexity
Time	$O(n)$
Space	$O(m)$

Where m is the number of distinct characters in the current window.

For a fixed-size character set such as standard ASCII, the auxiliary space can be considered $O(1)$.

12. Edge cases

Empty string

```
Input: ""  
Output: 0
```

The loop never runs, so the initial result 0 is returned.

One character

```
Input: "a"  
Output: 1
```

All characters identical

```
Input: "aaaa"  
Output: 1
```

The window repeatedly shrinks until only one `a` remains.

All characters unique

```
Input: "abcdef"  
Output: 6
```

The left pointer never moves.

Duplicate at the end

```
Input: "abcdca"  
Output: 4
```

Valid longest windows include `"abcd"` and `"bdca"`.

Spaces and punctuation

```
Input: "a b!a"
```

Spaces and punctuation are characters unless the problem says to normalize them.

Unicode characters

Python strings support Unicode characters:

```
Input: "अआइअ"  
Output: 3
```

The algorithm works without modification.

13. Important pitfalls

Pitfall 1: Updating the answer before restoring validity

Incorrect ordering:

```
frequency[character] += 1  
best = max(best, right - left + 1) # Window may contain duplicates.
```

Update the answer only after the window satisfies the invariant.

Pitfall 2: Shrinking only once

Incorrect:

```
if frequency[character] > 1:  
    frequency[s[left]] -= 1  
    left += 1
```

One removal may not restore validity. Use `while`.

Pitfall 3: Resetting the entire window

When a duplicate appears, do not restart from `right`.

That repeats work and can degrade toward $O(n^2)$.

Pitfall 4: Confusing substring and subsequence

A substring is contiguous.

For "pwwkew":

- "wke" is a substring.
- "pwke" is a subsequence, not a substring.

Pitfall 5: Maintaining stale counts

Every element removed from the left must have its frequency decremented.

Pitfall 6: Using the wrong validity condition

For this problem:

```
while frequency[character] > 1:
```

works because only the newly added character can create a fresh duplicate. In more complex problems, maintain a separate count such as:

```
number of duplicated characters
number of distinct characters
number of satisfied requirements
```

14. Alternative optimized solution using last-seen positions

Instead of storing counts, store the last index of each character.

When a duplicate appears inside the current window, jump `left` directly beyond its previous position.

```
def length_of_longest_substring_last_seen(s: str) -> int:
    last_seen: dict[str, int] = {}

    left = 0
    best_length = 0

    for right, character in enumerate(s):
        if character in last_seen:
            left = max(left, last_seen[character] + 1)

        last_seen[character] = right

        current_length = right - left + 1
        best_length = max(best_length, current_length)

    return best_length
```

The `max` is essential.

Consider:

```
s = "abba"
```

When processing the final `a`, its old index is before the current left boundary. Moving `left` backward would be incorrect.


```
left = max(left, last_seen[character] + 1)
```

prevents that error.

Comparison

Approach	State maintained	Advantage
Frequency map	Count of each character	Generalizes well to many window problems
Last-seen index	Most recent index	Jumps left directly
Set	Characters in window	Simple, but removes one by one

The frequency-map version is the best learning choice because the same pattern extends to:

- At most k distinct characters
- Longest repeating-character replacement
- Permutation in string
- Find all anagrams
- Minimum window substring

15. General variable-window templates

Longest valid window

```
def longest_valid_window(values: list[int]) -> int:
    left = 0
    best = 0
    state = {}

    for right, value in enumerate(values):
        # Add value to state.

        while window_is_invalid(state):
            outgoing = values[left]
            # Remove outgoing from state.
            left += 1

        best = max(best, right - left + 1)

    return best
```

Use this pattern for:

- Longest substring without duplicates
- Longest subarray with at most k distinct values
- Longest sequence with at most k replacements

Shortest valid window

```
def shortest_valid_window(values: list[int]) -> int:
    left = 0
    best = float("inf")
    state = {}

    for right, value in enumerate(values):
        # Add value to state.

        while window_is_valid(state):
            best = min(best, right - left + 1)

            outgoing = values[left]
            # Remove outgoing from state.
            left += 1

    return -1 if best == float("inf") else best
```

Use this pattern for:

- Minimum-size subarray sum
- Minimum window substring
- Smallest range satisfying a requirement

The key distinction is:

- **Longest valid:** shrink while invalid, then update.
- **Shortest valid:** shrink while valid, updating before each removal.

16. Interview explanation

A concise interview explanation would be:

“Because the problem asks for the longest contiguous substring satisfying a local constraint, I use a variable sliding window. A frequency map tracks characters in the current window. After adding the right character, I move the left boundary until that character is no longer duplicated. The invariant is that the resulting window contains each character at most once. Both pointers move only forward, so each character enters and leaves the window at most once, giving $O(n)$ time and $O(m)$ space.”

17. Final takeaway

For sliding-window problems, identify four things:

1. **What does the window represent?** A contiguous substring or subarray.
2. **What state must be maintained?** Sum, frequency map, distinct count, or another compact statistic.
3. **What makes the window valid or invalid?** This becomes the invariant.

4. When should the left pointer move? Until validity is restored or until further shrinking would lose validity.

For today's problem:

```
Window:
    s[left:right + 1]

State:
    character frequencies

Invariant:
    every character count <= 1

Shrink condition:
    newly added character count > 1

Answer:
    maximum valid window length
```